ASSIGNMENT

Simple Agent Function for the Vacuum-Cleaner World

1. Introduction

An **agent** is anything that perceives its *environment* through *sensors* and acts upon that environment through *actuators*. Mathematically, an agent's behaviour is described by the **agent function**, a mapping from the percept sequence (the complete history of everything the agent has perceived) to an action:

f : P* -> A

The agent function is an abstract mathematical description, whereas the **agent program** is the concrete implementation that runs on a physical architecture. The **vacuum-cleaner world** is a classic, deliberately simple toy problem (Russell and Norvig, *Artificial Intelligence: A Modern Approach*) used to explain these ideas clearly.

2. Description of the Vacuum-Cleaner World

The world consists of just two squares (locations), **A** and **B**. Each square may be **Clean** or **Dirty**. The vacuum agent is in exactly one square at any time.

Component	Description
Percepts	[Location, Status], e.g. [A, Dirty] or [B, Clean]. The agent senses only its current square.
Actions	**Left** (move to A), **Right** (move to B), **Suck** (clean current square), **NoOp** (do nothing).
Sensors	Location sensor and dirt sensor.
Actuators	Wheels (movement) and suction mechanism.
Total states	8 (2 agent positions x 2 x 2 dirt statuses).

3. PEAS Description

Element	Specification
Performance measure	One point per clean square at each time step (over a lifetime of, say, 1000 steps); optionally a penalty for each movement or energy used.
Environment	Two squares A and B with dirt that may appear; agent's location is known.
Actuators	Left, Right, Suck, NoOp.

Element	Specification
Sensors	Current location and whether it contains dirt.

4. The Simple Agent Function

A simple, sensible agent function is: **if the current square is dirty, suck; otherwise move to the other square.** It can be tabulated (partial table shown below):

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
[A, Clean], [B, Clean]	Left
[A, Clean], [B, Dirty]	Suck
...	...

Because the best action here depends only on the *current* percept, the table collapses to four rules and the agent needs no memory. Such an agent is called a **simple reflex agent**, using condition-action rules.

5. Agent Program (Pseudocode)

```
function REFLEX-VACUUM-AGENT([location, status])
        returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

Python implementation:

```
def reflex_vacuum_agent(percept):
    location, status = percept
    if status == "Dirty":
        return "Suck"
    elif location == "A":
        return "Right"
    else:
        return "Left"

# Simulation
world = {"A": "Dirty", "B": "Dirty"}
loc, score = "A", 0
for step in range(6):
    action = reflex_vacuum_agent((loc, world[loc]))
```

```
    if action == "Suck":   world[loc] = "Clean"
    elif action == "Right": loc = "B"
    elif action == "Left":  loc = "A"
    score += sum(1 for s in world.values() if s == "Clean")
    print(step + 1, action, loc, world, score)
```

6. Sample Execution Trace

Starting with both squares dirty and the agent in A:

Step	Percept	Action	New state	Score
1	[A, Dirty]	Suck	A clean, B dirty, at A	1
2	[A, Clean]	Right	A clean, B dirty, at B	2
3	[B, Dirty]	Suck	A clean, B clean, at B	4
4	[B, Clean]	Left	Both clean, at A	6
5	[A, Clean]	Right	Both clean, at B	8

7. Rationality and Analysis

A **rational agent** chooses, for each percept sequence, the action expected to maximise its performance measure, given the evidence from percepts and its built-in knowledge. Whether the above agent is rational depends on the performance measure and environment assumptions:

- With the measure 'one point per clean square per time step', and a known, fixed environment where clean squares stay clean, the agent **is rational**.
- If each movement costs a point, the agent is **not rational** once everything is clean, because it keeps moving back and forth. A better agent would choose **NoOp** when both squares are known to be clean (this requires some memory or internal state).
- If clean squares can become dirty again, the agent should re-inspect periodically, and if the geography is unknown it must explore and learn - features of more advanced agents.

8. Properties of the Environment

Property	Vacuum world
Observability	Partially observable (senses only its own square)
Agents	Single agent
Determinism	Deterministic (assuming actions always succeed)
Episodic / Sequential	Sequential
Static / Dynamic	Static (unless dirt appears randomly)
Discrete / Continuous	Discrete

9. Conclusion

The vacuum-cleaner world shows the core ideas of intelligent agents in a minimal setting: percepts, actions, the agent function as a mapping from percept sequences to actions, the agent program that implements it, and the performance measure that defines rational behaviour. The simple reflex agent works well in this small, fully specified world, but its limits highlight why model-based, goal-based, utility-based and learning agents are needed for more complex environments.

10. References

- Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*, Pearson (Chapter 2: Intelligent Agents).
- Standard course material on intelligent agents.